

2D to 3D Reconstruction — Complete Presentation Notes

Steps 1 through Frontend

STEP 1: Image Input & Loading

What Happens

The pipeline begins by loading a standard 2D image (JPG/PNG) using OpenCV.

Key Operations

- `cv2.imread()` loads the image in BGR format (OpenCV default).
- Immediately converted to RGB: `cv2.cvtColor(img, cv2.COLOR_BGR2RGB)`.
- Also converted to PIL Image format for compatibility with HuggingFace models.

Why BGR → RGB?

OpenCV stores images in BGR (Blue-Green-Red) by default for historical reasons.

All modern AI models and display libraries expect RGB.

Failing to convert causes blue and red channels to be swapped — the model sees wrong colors.

Hyperparameters

Parameter	Value	Effect
Image Format	JPG/PNG	Any standard format works
Color Space	RGB	Required by DepthAnything V2

STEP 2: Image Processing (script: 02_image_processing.py)

What Happens

The raw image is preprocessed to extract structural information and prepare a "guide image" for depth refinement.

Operations Performed

2a. Grayscale Conversion

```
gray = cv2.cvtColor(img, cv2.COLOR_BGR2GRAY)
```

- Reduces 3-channel RGB to 1-channel luminance.
- Removes color information, keeping only structural brightness.
- **Why:** Edge detection and depth refinement work better on luminance than color.

2b. Gaussian Blur (Noise Reduction)

```
blurred = cv2.GaussianBlur(gray, (5, 5), 0)
```

- Applies a 5×5 Gaussian kernel to smooth the image.
- **Formula:** Each pixel becomes a weighted average of its 5×5 neighbourhood.
- Weights follow a Gaussian distribution: $w(x,y) = \exp(-(x^2+y^2) / 2\sigma^2)$
- **Why:** Removes high-frequency noise before edge detection (Canny is sensitive to noise).

2c. Canny Edge Detection

```
edges = cv2.Canny(blurred, 50, 150)
```

- Detects structural edges using gradient magnitude + non-maximum suppression.
- **Two Thresholds:**
- Low threshold = 50: Edges weaker than this are discarded.
- High threshold = 150: Edges stronger than this are always kept.
- Edges between 50-150 are kept only if connected to a strong edge.
- **Why:** Edges mark object boundaries — critical for understanding scene structure.

2d. LAB Color Space (Advanced Guide)

When external grayscale is unavailable, the L channel from LAB color space is used:

```
lab = cv2.cvtColor(img, cv2.COLOR_RGB2LAB)
L, A, B = cv2.split(lab)
```

- **L channel** = Perceptual Luminance (matches human vision).
 - **Why LAB over simple grayscale:** LAB separates luminance from color (chrominance), giving a more perceptually accurate brightness map.
- Simple RGB-to-gray averages all channels equally, which can misrepresent brightness.

Hyperparameters Tested

Parameter	Values Tried	Final Value	Effect
Gaussian Kernel	(3,3), (5,5), (7,7)	*(5,5)*	Larger = more blur, loses edge detail
Canny Low Threshold	30, 50, 80	50	Lower = more edges detected
Canny High Threshold	100, 150, 200	150	Higher = only strong edges kept
Color Space	RGB-gray, LAB-L	LAB-L	LAB gives perceptually accurate luminance

Output

- output/gray_processed.jpg — Grayscale image used as guide for depth refinement.

STEP 3: Depth Estimation (script: 03_depth_estimation.py)

What Happens

The core AI model (DepthAnything V2) estimates a depth value for every pixel in the image.

Model: DepthAnything V2 (Small)

- **Architecture:** Vision Transformer (ViT) encoder + DPT decoder.
- **Training:** Pretrained on 1.5M+ images with pseudo-depth labels.
- **Output:** A single-channel "disparity map" where:
 - **Bright pixels = Close to camera**
 - **Dark pixels = Far from camera**
- **HuggingFace Model ID:** depth-anything/Depth-Anything-V2-Small-hf

Depth Map Normalization

```
depth_norm = (depth_raw - depth_min) / (depth_max - depth_min)
```

- Scales depth values to [0, 1] range.
- **Why:** Raw depth values are arbitrary floats. Normalizing makes them comparable across different images and usable as Z coordinates.

Grayscale-Guided Depth Refinement (Key Innovation)

After getting the raw depth map, we refine it using the Guided Filter:

Guided Filter Formula (He et al., ECCV 2010)

For each pixel i in a window ω_k :

```
q_i = a_k * I_i + b_k
```

Where:

- q = refined output (depth map)
- I = guide image (grayscale/L channel)
- $a_k = \text{cov}(I, p) / (\text{var}(I) + \epsilon)$ [edge-preserving coefficient]
- $b_k = \text{mean}(p) - a_k * \text{mean}(I)$ [offset]
- ϵ = regularization (prevents division by zero, controls smoothness)

Implementation

```
depth_refined = guided_filter(guide_image, depth_map_norm, radius=8, eps=0.01)
```

Hyperparameters Tested

Parameter	Values Tried	Final Value	Effect
Guided Filter Radius	4, 8, 16	**8**	Larger = smoother edges, loses detail
Epsilon (ϵ)	0.001, 0.01, 0.1	**0.01**	Larger = more smoothing, less edge preservation
Focal Length Scale	0.5, 0.8, 1.0	**0.8**	Controls field of view approximation
Depth Scale	500, 1000, 2000	**1000**	Scales depth values for Open3D
Depth Truncation	500, 1000	**1000**	Cuts off points beyond this depth

Why Guided Filter?

- Raw DepthAnything depth maps have "soft" edges — object boundaries are blurry.
- The grayscale image has sharp edges (real pixel boundaries).
- The Guided Filter transfers the sharpness from the grayscale guide to the depth map.
- Result: Depth map with sharper, more accurate object boundaries.

Multi-View Point Cloud Generation (3 Outputs)

From the same depth map, three point clouds are generated:

1. RGB Point Cloud

- Colors: Original image RGB values.
- Use: Photorealistic visualization.

2. Grayscale Point Cloud

- Colors: L channel (luminance only).
- Use: Structure analysis without color bias. Useful for scientific/architectural work.

3. Heatmap Point Cloud

- Colors: INFERNO colormap applied to depth values.
- Use: Depth visualization for debugging and education.
- **Why INFERNO over JET:**
 - Perceptually uniform (equal color steps = equal depth steps).
 - Colorblind-friendly.
 - Monotonically increasing luminance.
 - Industry standard for scientific visualization.
 - Reference: "A Better Default Colormap for Matplotlib" (Smith & van der Walt, 2015).

Pinhole Camera Back-Projection Formula

$$\begin{aligned} X &= (u - cx) * d / fx \\ Y &= (v - cy) * d / fy \\ Z &= d \end{aligned}$$

Where:

- (u, v) = pixel coordinates
- d = depth value
- fx, fy = focal lengths (approximated as image_width * focal_scale)
- (cx, cy) = principal point (image center)

Point Cloud Cleanup

```
cl, ind = pcd.remove_statistical_outlier(nb_neighbors=50, std_ratio=2.0)
```

- Removes "flying pixels" — isolated noisy points far from the main surface.
- **nb_neighbors=50:** Each point is compared to its 50 nearest neighbours.
- **std_ratio=2.0:** Points more than 2 standard deviations from mean distance are removed.

Coordinate Transform

```
pcd.transform([[1,0,0,0],[0,-1,0,0],[0,0,-1,0],[0,0,0,1]])
```

- Flips Y and Z axes to match standard 3D coordinate conventions.
 - Prevents the point cloud from appearing upside-down in viewers.
-

STEP 4: Point Cloud Visualization (script: 04_visualize_point_cloud.py)

What Happens

The generated .PLY point cloud files are loaded and displayed in an interactive Open3D window.

Viewer Controls

- Left-click + drag: Rotate
- Right-click + drag: Pan
- Scroll: Zoom

Initial Camera Setup

```
vis.get_view_control().set_front([0, 0, -1]) # Camera looks along -Z
vis.get_view_control().set_lookat([0, 0, 0]) # Looking at origin
vis.get_view_control().set_up([0, -1, 0]) # Y is up
vis.get_view_control().set_zoom(0.8) # Slight zoom out
```

What This Proves

- The depth map correctly encodes 3D structure.
 - The grayscale-guided refinement produces sharper point cloud edges.
 - The three views (RGB, Grayscale, Heatmap) demonstrate multi-modal output.
-

STEP 5: Mesh Generation (script: 05_mesh_generation.py)

What Was Tried: Poisson Surface Reconstruction

What Poisson Reconstruction Is

A mathematical algorithm that:

1. Takes point cloud + surface normals.
2. Solves a Poisson equation (PDE) to find a smooth implicit surface.
3. Extracts a watertight mesh (like inflating a balloon around the points).

Why It Was Abandoned

- For single-view images, only the front face exists.
- Poisson tries to "close" the back, creating blob/bubble artifacts.
- Smooths out sharp edges — loses fine detail.
- Requires accurate surface normals (hard to compute from monocular depth).

What Was Used Instead: Grid Meshing (Height-Map Tessellation)

How It Works

For every 4 neighbouring pixels (a quad), split into 2 triangles:

```
Triangle 1: (top-left, top-right, bottom-left)
Triangle 2: (top-right, bottom-right, bottom-left)
```

Why It's Better

- Speed: Runs in milliseconds on CPU.
- Sharpness: Preserves every pixel-level detail.
- UV Mapping: 3D grid matches 2D pixel grid exactly — texture maps with zero distortion.

Hyperparameters Tested

Parameter	Values Tried	Final Value	Effect
Downsample Factor	1, 2, 4, 8	4	Controls mesh density vs file size
Depth Scale	0.3, 0.5, 1.0, 500	0.5	Height of the 3D relief

STEP 6: Textured Mesh & Frontend (app.py)

UV Mapping Formula

```
u = x_pixel / (W - 1)
v = 1 - (y_pixel / (H - 1))
```

- $u, v \in [0, 1]$ — normalized texture coordinates.
- The 1 - on V flips the Y axis (image Y goes down, texture V goes up).

Vertex Colors (Final Method)

Instead of UV mapping + external texture file:

- Each vertex stores its own RGB color sampled directly from the image.
- Packed into a single .GLB file (no external .jpg needed).
- Advantage: Works in any browser, no file path issues.

GLB Format

- GL Transmission Format — the "JPEG of 3D".
- Single binary file containing geometry + colors/textures.
- Standard format for web, AR, and VR applications.

Key Design Decisions

Decision	Reason
GLB over OBJ	GLB is self-contained (no external texture files)
Vertex Colors over UV Texture	Avoids file path issues in browser security sandbox
Downsample Factor = 4	Keeps GLB file under 5MB for fast loading

Decision	Reason
doubleSided=True	Makes back face visible when user rotates model
clear_color=[1,1,1,1]	White background for contrast
X-axis flip on geometry	Makes mesh face camera by default
Horizontal image pre-flip	Cancels mirror effect from X-axis geometry flip

Pipeline Flow in the App

User uploads image
 → DepthAnything V2 estimates depth map
 → Grid Meshing creates triangulated surface
 → Vertex colors sampled from original image
 → Trimesh exports as .GLB (single binary file)
 → Gradio Model3D renders it interactively in browser

MULTI-VIEW FUSION (What It Is & Why Not Used)

What Multi-View Fusion Is

Technique of combining depth/geometry from multiple camera angles:

1. Take N photos of the same object from different angles.
2. Run depth estimation on each.
3. Register (align) all point clouds using camera poses (from SLAM or SfM).
4. Fuse overlapping regions, averaging depth values.
5. Run Poisson on the merged point cloud → complete watertight mesh.

Why Not Used

- Requires multiple images + camera calibration data.
- This project is monocular (single image input) by design.
- The Transformer approach (TripoSR) is the modern replacement:
 it learns to hallucinate missing views from a single image using a trained prior.

How Poisson + Grid Meshing Could Work Together

In an advanced pipeline:

1. Grid Mesh → Fast, detailed front face.
2. Poisson → Smooth, closed back face (from estimated normals).
3. Merge → Stitch front (Grid) + back (Poisson) into one watertight mesh.

This is essentially what NeRF and TripoSR do internally.

QUICK ANSWER SHEET (For Q&A)

Question	Answer
Why is the back empty?	Monocular depth (2.5D). Cannot hallucinate unseen geometry. Generative models (TripoSR) needed for that.
Why Grid Meshing over Poisson?	Best balance of speed and texture accuracy. Poisson creates blob artifacts for single-view images.
Is this real-time?	Yes. Local pipeline runs in under 3 seconds on CPU.
What is the .GLB file?	Standard web 3D format bundling geometry + colors in one binary file.
Why DepthAnything V2?	State-of-the-art monocular depth estimation. Pretrained on 1.5M+ images. Runs on CPU.
What is the Guided Filter for?	Sharpens depth map edges using the grayscale image as a structural guide.
Why INFERNO colormap?	Perceptually uniform, colorblind-friendly, scientifically standard.
Why 3 point cloud outputs?	Multi-modal validation: RGB (visual), Grayscale (structural), Heatmap (depth analysis).